

Full-Duplex UART

Design, Implementation & Verification

Martin Mounir Fayez

SystemVerilog RTL Project | Simulated in ModelSim

1. Project Overview

This project implements a full-duplex UART (Universal Asynchronous Receiver/Transmitter) in SystemVerilog, capable of serializing and deserializing 8-bit parallel data with optional even/odd parity checking. The design is split into a transmit (TX) datapath and a receive (RX) datapath, each built from small, single-purpose modules coordinated by their own finite state machine.

The system was verified using a custom loopback testbench in ModelSim, connecting the TX output directly to the RX input and confirming that bytes sent on the transmit side are correctly reconstructed on the receive side, with parity handled correctly in both modes.

2. Architecture

2.1 Transmit (TX) Path

The `uart_tx` top module coordinates four sub-blocks:

- **fsm** — sequences the frame through START, DATA, PARITY, and STOP states and drives the output mux select line.
- **Serializer** — shifts the 8-bit parallel input out one bit at a time on each clock.
- **Parity_bit_calc** — computes even or odd parity over the input byte.
- **mux** — selects between the start bit, serial data, parity bit, and stop bit to drive the single-wire TX output.

2.2 Receive (RX) Path

The `uart_rx` top module mirrors the TX side with three sub-blocks:

- **edge_detector** — detects the falling edge of the incoming line to identify the start bit.
- **fsm_rx** — sequences DATA, PARITY, and STOP states in step with the incoming bitstream.
- **deserializer** — shifts incoming bits into an 8-bit parallel register.
- **parity_check** — recomputes parity on the received byte and flags a mismatch.

Both FSMs are clocked and reset synchronously to the shared system clock, so no explicit baud-rate generator is required in this version — one clock cycle corresponds to one bit period.

3. Verification Environment

A dedicated loopback testbench (**uart_loopback_tb**) instantiates both **uart_tx** and **uart_rx**, wiring the TX serial output directly to the RX serial input. The testbench drives four bytes through the system under different parity configurations:

Byte	Binary	Parity Mode
0xA5	1010 0101	Disabled
0x3C	0011 1100	Disabled
0x5A	0101 1010	Even
0xF0	1111 0000	Odd

Between each byte, the testbench inserts idle clock cycles so that individual frames are clearly separated in the resulting waveform — useful for a readable, presentable capture.

4. Compilation

All ten RTL modules were added to a ModelSim project and compiled together with no errors, confirming the design is syntactically correct and fully elaborates as one hierarchy.

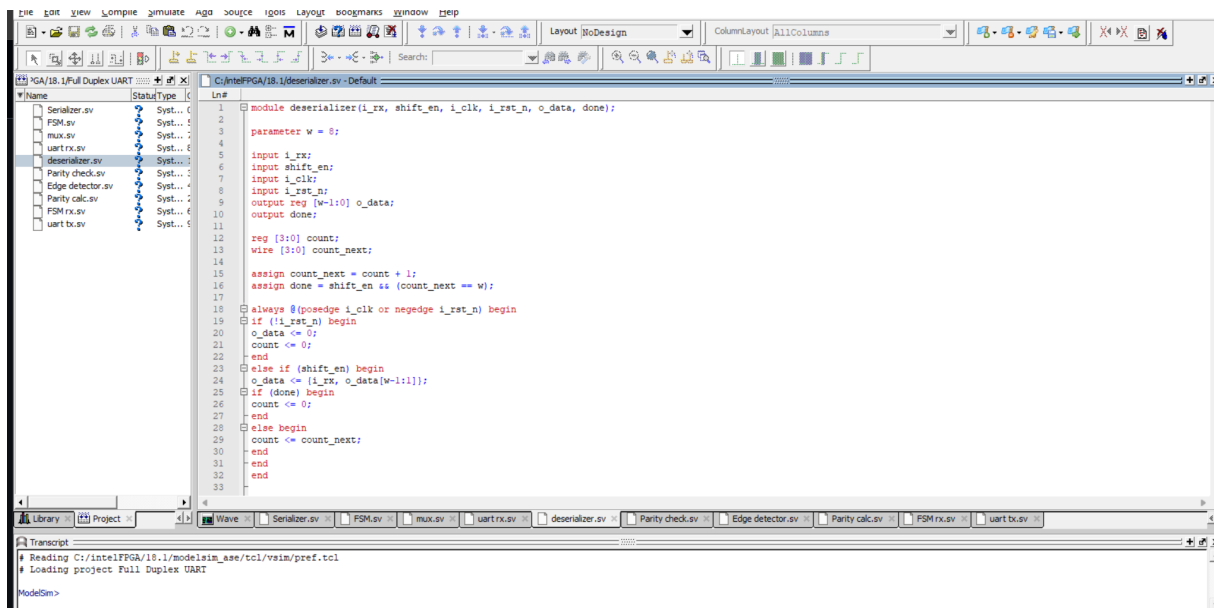


Figure 1 — ModelSim project with all UART modules compiled successfully.

5. Simulation Results

The waveform below shows one full run of the loopback testbench. The **tx_data** signal shows each byte as it is loaded into the transmitter; the **line** signal is the actual serial bitstream leaving TX and entering RX; and **rx_data** shows the byte reconstructed by the receiver.

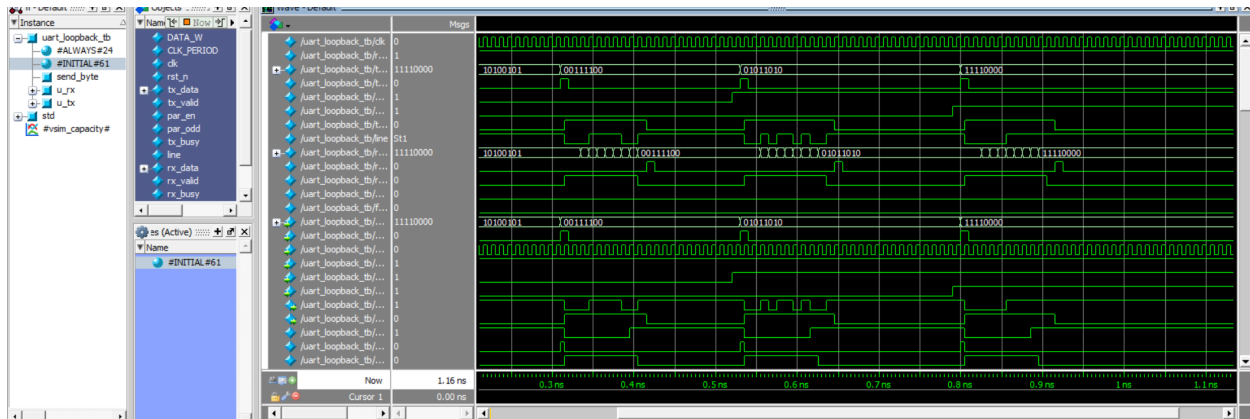


Figure 2 — Loopback waveform: transmitted bytes are correctly recovered on the receive side, including under even and odd parity.

For every transmitted byte, the value observed on **rx_data** after the frame completes is bit-for-bit identical to the value driven on **tx_data**, for both the parity-disabled and parity-enabled cases. This confirms correct operation of serialization, parity generation, parity checking, and deserialization across the full TX → RX chain.

6. Conclusion

The full-duplex UART design successfully transmits and receives 8-bit data frames with optional parity, verified end-to-end through a self-checking loopback simulation in ModelSim. The modular structure — separate serializer, deserializer, parity, and FSM blocks on each side — keeps the datapath and control logic cleanly separated and easy to extend, for example with a baud-rate generator for true asynchronous operation in a future revision.